

深圳大学实验报告

课程名称： 中间件技术

实验项目名称： 实验 4 应用 JNI 跨语言混合编程

学院： 计算机与软件学院

专业： 软件工程

指导教师： 张洋

报告人： 学号： 班级：

实验时间： 2026 年 6 月 3 日

实验报告提交时间： 2026 年 6 月 19 日

教务部制

一、实验目的与要求:

通过编程问题的解决,直观的感受、体会和学用 Java JNI 解决异构语言编程问题,在 Java 程序中调用 C++程序。

https://blog.csdn.net/qq_41969790/article/details/106083780?utm_medium=distribute.pc_relevant.none-task-blog-2%7Edefault%7EBlogCommendFromBaidu%7Edefault-6.no_search_link&depth_1-utm_source=distribute.pc_relevant.none-task-blog-2%7Edefault%7EBlogCommendFromBaidu%7Edefault-6.no_search_link

二、实验内容

内容:解决以下异构语言编程互操作问题

在某软件开发项目中需要混合编程,其中,算法程序使用 C/C++编写,而算法服务器是用 Java 编写,使用 Java JNI 来实现 C/C++算法部分代码的调用。

编程过程分如下几步:

1.创建 Java 工程

- (1) 新建 Java 工程
- (2) 编写对应 C/C++程序的 Java 对象类

2.编译制作 dll 动态库

- (1) 创建 dll 工程
- (2) 编写 C/C++源文件
- (3) 生成 DLL 动态链接库文件

3.Java 程序调用

详细编程过程指导,请参考实验 4 应用 JNI 实现 Java 调用 C/C++代码编程指导.PDF

报告写作。要求:主要思路有明确的说明,重点代码有详细的注释,行文逻辑清晰可读性强,报告整体写作较为专业。**(20 分)**

说明:

- (1) 本次实验课作业满分为 100 分,占总成绩的比例(待定)。
- (2) 报告正文:请在指定位置填写,本次实验**需要提交实验报告+源码**,其中,源码按工程包(或文件夹)路径不变压缩成一个.rar(或.zip)文件提交,实验报告不要压缩进压缩包文件。
- (3) 个人信息:WORD 文件名中的“姓名”、“学号”,请改为你的姓名和学号;实验报告的首页,请准确填写“学院”、“专业”、“报告人”、“学号”、“班级”、“实验报告提交时间”等信息。
- (4) 提交方式:截至时间前,请在 Blackboard 平台中提交。
- (5) 发现雷同,所有雷同者该次作业记零分。
- (6) 期末考试阶段补交无效。

三、实验过程及结果

请将源码，实验过程及运行结果简述于此。

1.创建 Java 工程

①在 src 文件夹下创建 jni 包，新建 Demo 类。

```
1 package jni;
2
3
4 public class Demo { 3个用法
5
6     // Load the native library when this class is loaded
7     static {
8         System.loadLibrary( libname: "JniDemo");
9     }
10
11     public native int[] quickSort(int[] arr); 1个用法
12     public native long fibonacci(int n); 1个用法
13 }
14
```

该类通过 `static { System.loadLibrary("JniDemo"); }` 静态块加载 C++编译生成的 JniDemo.dll 动态链接库，并使用 `native` 关键字声明两个本地方法 —— `quickSort`（快速排序）和 `fibonacci`（斐波那契数列），这两个方法不提供 Java 实现体，而是由 C++ 端实现。

②生成 .class 文件和 JNI 头文件

```
PS E:\Personal\CSSE_中间件技术\实验4\Ex4> javac -h cpp src/jni/Demo.java -d out
```

在项目根目录执行命令 `javac -h cpp src/jni/Demo.java -d out`，先编译 Demo.java 生成 .class 字节码文件；再使用 `javac -h` 参数生成 C/C++端需要的 JNI 头文件 jni_Demo.h。该头文件由 JDK 自动生成，包含 `Java_jni_Demo_quickSort` 和 `Java_jni_Demo_fibonacci` 两个函数声明，作为 Java 层与 C++ 层的桥梁。

```
1 #include <jni.h>
2
3 #ifndef _Included_jni_Demo
4 #define _Included_jni_Demo
5 #ifdef __cplusplus
6 extern "C" {
7 #endif
8
9 JNIEXPORT jintArray JNICALL Java_jni_Demo_quickSort
10 (JNIEnv *, jobject, jintArray);
11
12
13 JNIEXPORT jlong JNICALL Java_jni_Demo_fibonacci
14 (JNIEnv *, jobject, jint);
15
16 #ifdef __cplusplus
17 }
18 #endif
19 #endif
20
```

2.编译制作 dll 动态库

①创建 dll 工程

创建 cpp 目录用于存放 C++ 工程文件，并编写 CMakeLists.txt 构建配置文件。该文件指定了 CMake 最低版本、C++17 标准、JNI 头文件的包含路径（JDK 的 include 和 include/win32 目录），以及构建目标名为 JniDemo 的动态链接库（.dll）。

```
1 cmake_minimum_required(VERSION 3.10)
2 project(JniDemo)
3
4 set(CMAKE_CXX_STANDARD 17)
5
6 # JNI headers
7 include_directories("E:/Programs/JDK/include")
8 include_directories("E:/Programs/JDK/include/win32")
9
10 # Build the DLL
11 add_library(JniDemo SHARED Demo.cpp)
12
13 # Link required Windows libraries
14 if(WIN32)
15     target_link_libraries(JniDemo PRIVATE jvm)
16 endif()
17
```

②编写 C/C++ 源文件

在 cpp 目录下编写 Demo.cpp，实现 jni_Demo.h 中声明的两个函数：

```
1 #include "jni_Demo.h"
2 #include <cstdlib>
3 #include <cstring>
4 #include <new>
5
6 static inline void swap(jint& a, jint& b) {
7     jint t = a;
8     a = b;
9     b = t;
10 }
11
12 static void quickSortRange(jint* a, int left, int right) {
13     if (left >= right) return;
14     int i = left, j = right;
15     jint pivot = a[(left + right) / 2];
16     while (i <= j) {
17         while (a[i] < pivot) ++i;
18         while (a[j] > pivot) --j;
19         if (i <= j) {
20             swap(a[i], a[j]);
21             ++i;
22             --j;
23         }
24     }
25     quickSortRange(a, left, j);
26     quickSortRange(a, i, right);
27 }
```

```

29 static void quicksort(jint* a, int n) {
30     quickSortRange(a, 0, n - 1);
31 }
32
33 JNIEXPORT jintArray JNICALL Java_jni_Demo_quickSort
34 (JNIEnv *env, jobject /* obj */, jintArray arr)
35 {
36     jsize len = env->GetArrayLength(arr);
37
38     // Get elements from Java array into native buffer
39     jint *buf = env->GetIntArrayElements(arr, nullptr);
40     if (buf == nullptr) return nullptr; // OutOfMemoryError already thrown
41
42     // Allocate working copy
43     jint* copy = new (std::nothrow) jint[len];
44     if (copy == nullptr) {
45         env->ReleaseIntArrayElements(arr, buf, JNI_ABORT);
46         return nullptr;
47     }
48     for (jsize i = 0; i < len; ++i) copy[i] = buf[i];
49
50     env->ReleaseIntArrayElements(arr, buf, JNI_ABORT);
51
52     // Sort
53     quicksort(copy, len);
54
55     // Create result Java array
56     jintArray result = env->NewIntArray(len);
57     env->SetIntArrayRegion(result, 0, len, copy);
58
59     delete[] copy;
60     return result;
61 }

```

```

63 JNIEXPORT jlong JNICALL Java_jni_Demo_fibonacci
64 (JNIEnv *env, jobject /* obj */, jint n)
65 {
66     if (n < 0) {
67         env->ThrowNew(env->FindClass("java/lang/IllegalArgumentException"),
68             "n must be >= 0");
69         return 0;
70     }
71     if (n == 0) return 0;
72     if (n == 1) return 1;
73
74     jlong a = 0, b = 1;
75     for (jint i = 2; i <= n; ++i) {
76         jlong c = a + b;
77         a = b;
78         b = c;
79     }
80     return b;
81 }
82

```

quicksort 算法：采用 Hoare 划分方案的经典快排，选取中间元素作为基准（pivot），原地递归排序。JNI 层先用 GetIntArrayElements 从 Java 数组取出数据，调用快排后，用 NewIntArray 和 SetIntArrayRegion 创建新的 Java 数组返回。

fibonacci 算法：采用迭代法计算第 n 个斐波那契数，F(0)=0, F(1)=1。对于非法输入（n<0），通过 JNI 的 ThrowNew 机制抛出 IllegalArgumentException。

③生成 DLL 动态链接库文件

使用 MinGW 的 g++ 编译器，以 -shared 参数将 Demo.cpp 编译为 64 位 Windows 动态链接库 JniDemo.dll，输出到 lib 目录下。编译时通过 -I 参数指定 JNI 必需的 JDK include

路径。

命令: `g++ -shared -o lib/JniDemo.dll cpp/Demo.cpp`

`-I"E:/Programs/JDK/include" -I"E:/Programs/JDK/include/win32" -std=c++17 -O2`

生成 lib/JniDemo.dll (260KB, x86-64 PE32+ 格式)

3.Java 程序调用

编写 Main.java 测试程序, 创建 Demo 对象后分别调用 quickSort 和 fibonacci 两个 native 方法。运行时通过 `Djava.library.path=lib` 参数指定 DLL 的搜索路径, JVM 加载 JniDemo.dll 后即可成功调用 C++ 端实现的算法。

```
1 > import ...
2
3
4
5 public class Main {
6     public static void main(String[] args) {
7         System.out.println("=== JNI Demo: Java calling C++ algorithms ===\n");
8
9         Demo demo = new Demo();
10
11         // ---- Test 1: quickSort ----
12         int[] unsorted = {42, 13, 7, 99, 23, 4, 87, 56, 31, 78};
13         System.out.println("Test 1 - quickSort");
14         System.out.println("  Input:  " + Arrays.toString(unsorted));
15
16         int[] sorted = demo.quickSort(unsorted);
17         System.out.println("  Output: " + Arrays.toString(sorted));
18
19         // Verify original array is unchanged (we return a new sorted copy)
20         boolean originalIntact = Arrays.equals(unsorted,
21         |         new int[]{42, 13, 7, 99, 23, 4, 87, 56, 31, 78});
22         System.out.println("  Original array unchanged: " + originalIntact);
23
24         // ---- Test 2: fibonacci ----
25         System.out.println("\nTest 2 - fibonacci");
26         int[] fibInputs = {0, 1, 2, 5, 10, 20, 30, 40, 50};
27         for (int n : fibInputs) {
28             long result = demo.fibonacci(n);
29             System.out.printf("    fibonacci(%2d) = %d\n", n, result);
30         }
31
32         System.out.println("\n=== All tests passed! ===");
33     }
34 }
35
```

```
PS E:\Personal\CSSE_中间件技术\实验4\Ex4> java -cp out "-Djava.library.path=lib" Main
=== JNI Demo: Java calling C++ algorithms ===

Test 1 - quickSort
  Input:  [42, 13, 7, 99, 23, 4, 87, 56, 31, 78]
  Output: [4, 7, 13, 23, 31, 42, 56, 78, 87, 99]
  Original array unchanged: true

Test 2 - fibonacci
  fibonacci( 0) = 0
  fibonacci( 1) = 1
  fibonacci( 1) = 1
  fibonacci( 2) = 1
  fibonacci( 5) = 5
  fibonacci(10) = 55
  fibonacci(20) = 6765
  fibonacci(30) = 832040
  fibonacci(40) = 102334155
  fibonacci(50) = 12586269025

=== All tests passed! ===
```

四、实验总结与体会

(写写感想、建议等)

本次实验围绕 JNI 跨语言混合编程完成 Java 调用 C++ 底层算法的完整流程，完整实现了从 Java 本地方法声明、JNI 头文件生成、C++ 动态链接库编译，到 Java 加载 dll 调用原生函数的全链路开发。

通过本次实验，我加深了对 JNI 跨语言交互核心原理的理解，掌握了 Java 与 C++ 数据类型转换与内存管理的相关知识，并学会动态链接库编译与运行环境配置。

本次实验完整落地 JNI 混合编程流程，从理论概念落地到可运行项目，清晰掌握了 Java 调用 C++ 的完整开发规范、数据交互、内存管控与异常处理，同时也认识到跨语言开发的复杂性。本次实验让我对中间件跨语言交互、异构系统集成有了更直观的理解，为后续学习分布式服务、高性能中间件底层交互打下基础。

五、成绩评定及评语

1.指导老师批阅意见：

2.成绩评定：

指导教师签字：张洋
2026 年 月 日